

RTL for AXI4 (Master & Slave) to PCIe 6 Link

SARCathon 2026 — Semiconductor Challenge

Stage 1 Technical Report — RTL Design & Functional Verification

Team Members:	Hemal Dedhia Raj Dwivedi Sayandeep Das
Institution:	IIT Bhilai
Date:	6 th September, 2026
Repository:	github.com/sayandeedas30/semicon

1. Problem Understanding and Selected Implementation Scope

The challenge asks for a synthesizable RTL bridge that carries data from an AXI4-Stream producer, through an on-chip interconnect and link layer, out to a PCIe-facing PHY boundary. The reference architecture defines four layers: an AXI4-Stream ingress/egress interface, an interconnect layer (central crossbar with dynamic arbitration), a link layer (packetization with FEC/CRC), and a PCIe protocol-to-PHY layer (multi-lane striping, Gray coding, PAM4 mapping).

Per the challenge's own evaluation philosophy, a smaller, correctly designed and thoroughly verified implementation is preferable to a larger, unverified one. Our team scoped Stage 1 accordingly: rather than attempting a full N-input crossbar or a production-grade FEC scheme, we implemented a reduced but representative instance of every layer in the reference architecture, verified each in isolation, and then verified the complete chain end-to-end in a single integrated simulation.

1.1 Scope Implemented in Stage 1

- AXI4-Stream ingress and egress interface (1-deep buffered pass-through with full valid/ready handshaking)
 - Interconnect layer: 2-input, 1-output crossbar with round-robin arbitration and packet-boundary-safe grant switching
- Link layer: packetizer bundling data into 256-bit flits with an XOR-based running CRC over an 8-flit window
- PCIe protocol-to-PHY layer: 4-lane striping, 2-bit Gray coding, and PAM4 symbol mapping
- A protocol/width adapter reconciling the AXI4-Stream domain (512-bit, tvalid/tready) with the link layer's domain (256-bit, valid-only, no backpressure)
- A complete, simulated, end-to-end system: two external AXI4-Stream inputs down to four PHY-facing symbol lanes

1.2 Explicitly Out of Scope for Stage 1

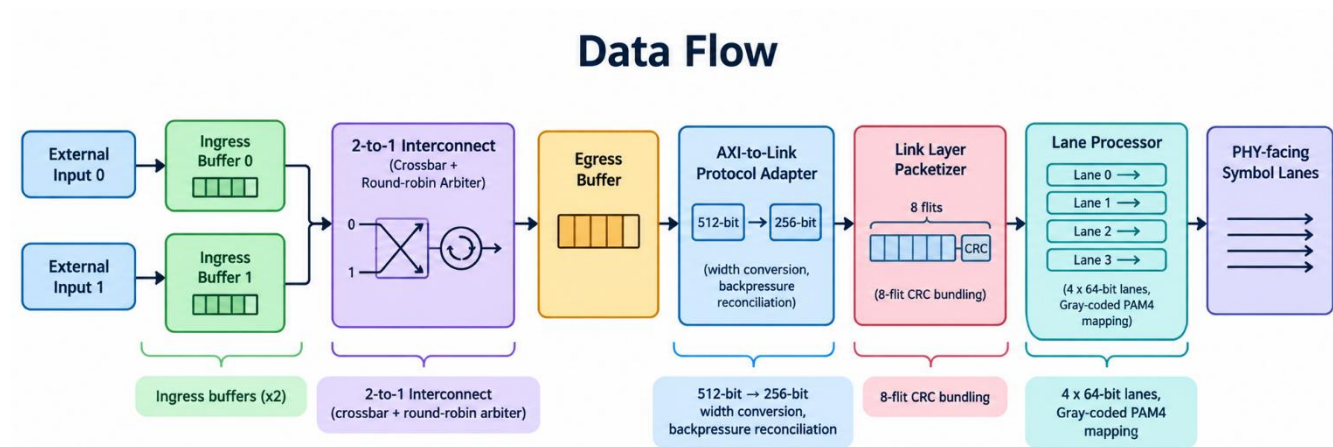
- General N x N crossbar — we implemented 2 inputs to 1 output as a representative, verifiable instance of the arbitration problem
- Production-grade Forward Error Correction — the link layer implements a simplified XOR-based checksum standing in for the FEC/CRC concept, not a full CRC-32 polynomial or Reed-Solomon FEC
- Analog SerDes / PHY — the design stops at the parallel symbol bus handed to the PHY, as specified in the reference block diagram

- Synthesis, timing closure, and GDSII generation — reserved for Stage 2 per the challenge's suggested technical flow
- FPGA prototyping (bonus deliverable) — not attempted in Stage 1

2. Block-Level Architecture and Design Methodology

The implemented system mirrors the reference block diagram, with two AXI4-Stream ingress ports feeding a shared interconnect, whose single merged output is adapted into the link layer's packetization and CRC pipeline, and finally into the PCIe-PHY layer's lane striping and PAM4 mapping stage.

2.1 Data Flow



2.2 Design Methodology

Development proceeded bottom-up and incrementally, verifying each module in isolation before composing it with its neighbors:

- Understand the block diagram and derive the exact signal-level interface for each layer before writing any RTL
- Implement the smallest correct version of each module first (e.g. a 1-deep buffer before considering deeper FIFOs)
- Write a self-checking, directed testbench for every module, covering reset, normal operation, backpressure, and boundary conditions specific to that module
- Confirm correct behavior via simulation waveforms in GTKWave before integrating with any other module
- Compose modules into progressively larger integration wrappers (module pair → subsystem → full system), each with its own integration-level test

3. RTL Design Description

3.1 AXI4-Stream Ingress / Egress (`axi_stream_ingress.v`, `axi_stream_egress.v`)

Both modules implement an identical 1-deep buffered pass-through primitive with full AXI4-Stream valid/ready handshaking (512-bit tdata, 64-bit tkeep, tlast, tvalid, tready). A single storage register holds one in-flight word; the module presents tready as either "buffer empty" or "buffer full but draining this same cycle" (a same-cycle swap), which allows back-to-back transfers with no wasted bubble cycle. Ingress and egress are functionally the same primitive, instantiated at opposite ends of the on-chip pipeline; only the external connections at each boundary differ (ingress faces the external producer, egress faces the next internal stage).

3.2 Interconnect (`axi_stream_interconnect.v`)

A 2-input, 1-output crossbar. The data path is a plain combinational mux selecting between the two inputs based on a 1-bit grant register; each input's tready is gated so only the currently granted input may transfer. The grant register updates on two conditions: (1) the granted input has just completed a packet (a tlast transfer) and the other input is requesting, or (2) the granted input is currently idle (not asserting tvalid at all) while the other input is requesting. The second condition was added after a design review identified that without it, a permanently idle granted input would starve the other input indefinitely, since idle inputs never produce a tlast to trigger a switch. Critically, the grant never switches mid-packet under any other condition, preventing interleaving of two different packets on the shared output.

3.3 Integration Wrappers (`axi_stream_top.v`, `system_top.v`)

`axi_stream_top.v` instantiates two ingress buffers into the interconnect and a single egress buffer on its output, exposing two external AXI4-Stream inputs and one external AXI4-Stream output. `system_top.v` composes `axi_stream_top.v` with the protocol adapter and the link/PHY subsystem into the complete Stage 1 chain, reconciling both a naming convention difference (`aclk/aresetn` versus `clk/rst_n` between the two sub-teams' modules) and a protocol difference (full AXI4-Stream handshaking versus a valid-only interface) at a single, clearly documented boundary.

3.4 Protocol Adapter (`axi_to_link_bridge.v`)

Since the link layer provides no backpressure signal at all, this adapter cannot simply pass tready through — it must independently track its own draining state and only present tready to the AXI side while idle. A 3-state FSM (IDLE, SEND_LOW, BUBBLE) accepts one 512-bit AXI4-Stream word, splits it into two sequential 256-bit halves for the link layer, and holds off accepting the next word until both halves have been sent. This is a deliberate simplicity-over-throughput trade-off appropriate for Stage 1 scope.

3.5 Link Layer Packetizer (`link_layer_packetizer.v`)

Bundles incoming 256-bit flits into groups of eight, computing a running XOR-based checksum across the first seven flits and combining it with the eighth flit to stand in for the FEC/CRC concept specified in the reference

architecture. This is a simplified error-detection scheme rather than a production CRC polynomial, appropriate to the reduced scope of Stage 1.

3.6 Gray/PAM4 Mapping and Lane Processing (gray_pam4_mapper.v, lane_processor.v, pcie_phy_layer.v)

gray_pam4_mapper.v implements the 2-bit-to-2-bit Gray-coded PAM4 symbol mapping table specified in the reference architecture. lane_processor.v instantiates this mapper 32 times to process a full 64-bit lane.

pcie_phy_layer.v applies this across four parallel 64-bit lanes, registering the output and holding the last valid symbols whenever no new data is presented, completing the striping described in the reference block diagram.

4. Interface Description and Key Design Decisions

4.1 AXI4-Stream Signal Set

Signal	Width	Direction	Purpose
tdata	512 bits	producer -> consumer	Payload
tkeep	64 bits	producer -> consumer	Byte-valid mask
tvalid	1 bit	producer -> consumer	Producer has valid data
tready	1 bit	consumer -> producer	Consumer can accept data
tlast	1 bit	producer -> consumer	Marks final word of a packet

4.2 Key Design Decisions

- **Reused a single buffer primitive for ingress and egress** rather than writing two separate implementations, since both solve the identical problem (a 1-deep elastic buffer with valid/ready handshaking) at opposite ends of the pipeline.
- **No storage buffer inside the interconnect** — since its inputs and output are already buffered by neighboring modules, the interconnect only needs a combinational mux and a grant register, not its own storage.
- **Grant may only switch at a packet boundary, or when the granted input is fully idle** — this is the single correctness rule that prevents word-level interleaving of two different packets on the shared output.
- **The AXI-to-link protocol adapter absorbs all boundary mismatches** (width, naming convention, and backpressure semantics) at one clearly identified point in the design, rather than each module having to accommodate the other's conventions directly.
- **AXI4-Stream semantics were retained on the internal bus between the interconnect and the link layer**, rather than immediately stripping down to a bespoke minimal bus, favoring implementation and verification simplicity for Stage 1.

5. Verification Strategy and Test Scenarios

Every module was verified with a self-checking, directed testbench before being integrated with any neighboring module, following the principle that a smaller, thoroughly verified design is preferable to a larger, unverified one. Each testbench uses a pass/fail counter driven by automated equality checks rather than requiring manual waveform inspection to determine correctness, though waveforms were also captured and inspected for every test as a secondary confirmation.

5.1 Ingress / Egress Test Scenarios

- **Reset behavior** — buffer correctly empty immediately after reset

- Simple single-word transfer with no stalls
- Backpressure — downstream not ready, confirms upstream is correctly stalled with no data corruption ●
Back-to-back transfer (buffer full and draining the same cycle new data arrives) — confirms no bubble cycle is introduced
- tlast propagation through the buffer

5.2 Interconnect Test Scenarios

- Single-input passthrough on each of the two inputs independently
- Contention — both inputs requesting simultaneously, confirming the non-granted input's tready is correctly held low
- Mid-packet hold — confirms the grant does not switch even when the other input is requesting, as long as the granted input's packet has not reached tlast
- Grant switch immediately following a completed tlast transfer when the other input is requesting

5.3 Link Layer and PHY Test Scenarios

Verified independently by [teammate], covering reset, an 8-flit burst exercising the full CRC window, and confirmation that the PHY layer correctly holds its output steady when no new valid data is presented.

5.4 Integration-Level Test Scenarios

- Ingress -> Egress direct connection: a single word proven to cross both buffers unchanged
- Full axi_stream_top chain (2x ingress -> interconnect -> egress): both a simple passthrough and a full contention/grant-switch scenario, verified end-to-end
- Full system_top chain: four 512-bit words driven into the external AXI4-Stream input, correctly producing 8 flits at the link layer, triggering one complete CRC cycle, and yielding four independently-computed-and-matched Gray/PAM4 lane symbol values

6. Simulation Waveforms and Result Snapshots

6.1 Ingress Module — Full Test Suite

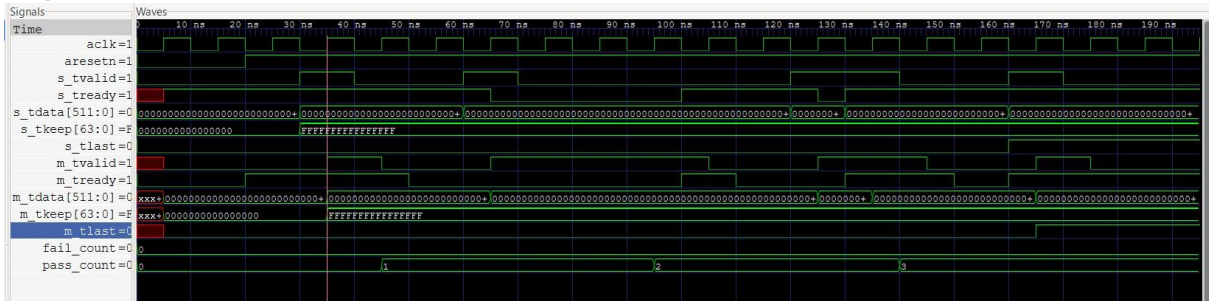


Figure 1: axi_stream_ingress — reset, simple transfer, backpressure, back-to-back swap, and tlast propagation. pass_count reaches 3 of 3 checked assertions with 0 fails (additional PASS/FAIL lines are printed to the simulation log for every scenario).

6.2 Ingress Module — Back-to-Back Swap Detail

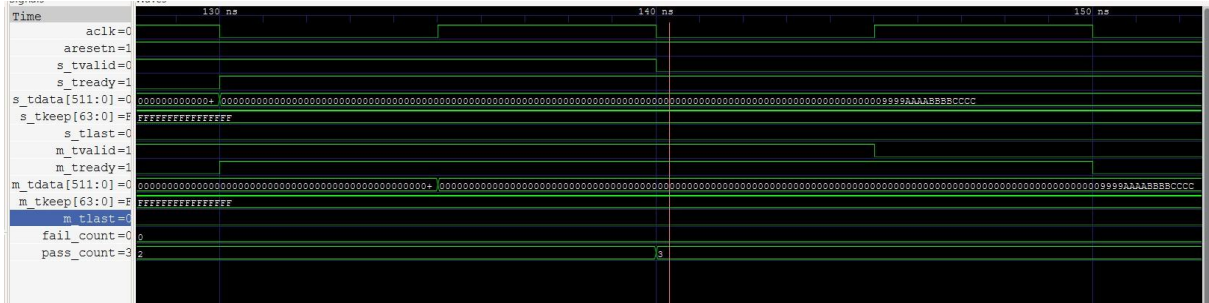


Figure 2: Zoomed view of the back-to-back swap case. m_tvalid remains continuously high across the transition between the two data words, confirming no bubble cycle is introduced when the buffer is full and draining on the same cycle a new word arrives.

6.3 Interconnect Module — Full Test Suite

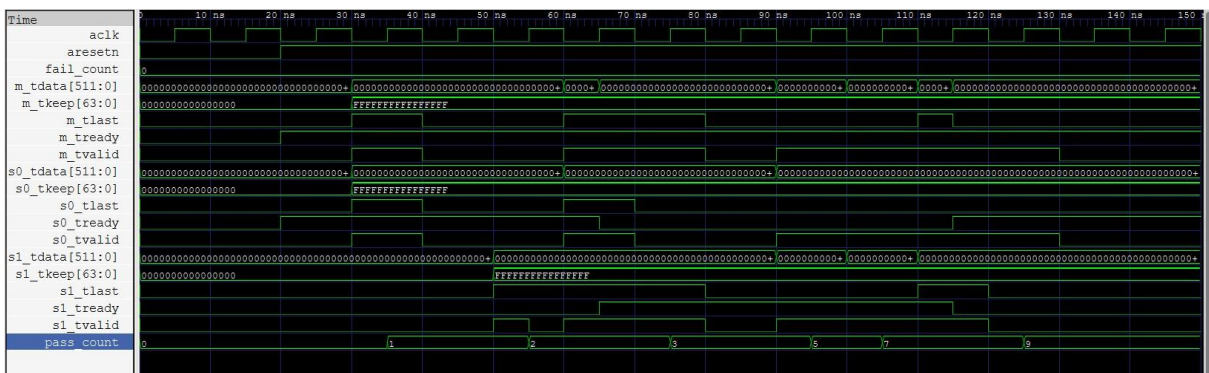


Figure 3: axi_stream_interconnect — single-input passthrough, contention gating, mid-packet grant hold, and grant switch after tlast. pass_count reaches 9 of 9 checks with 0 fails.

6.4 axi_stream_top Integration — Two Ingress Ports, Interconnect, Egress

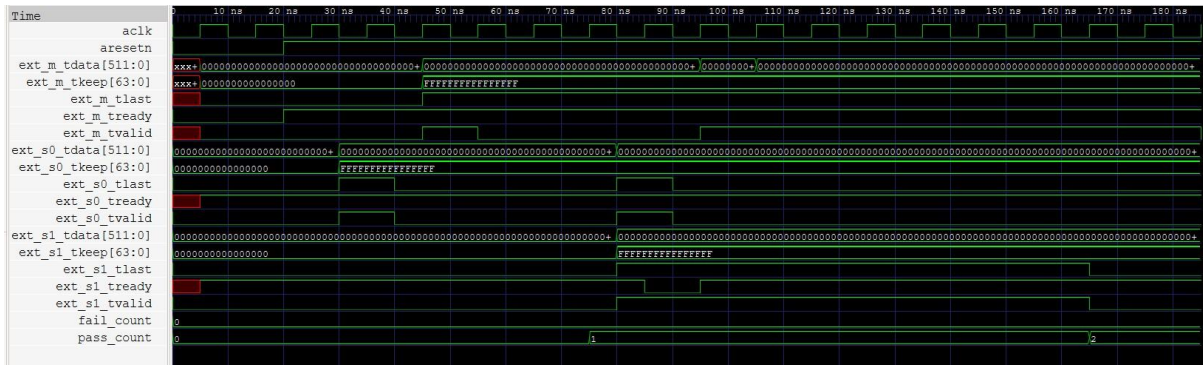


Figure 4: Full axi_stream_top chain test. Both external inputs drive data through their respective ingress buffers, the interconnect correctly arbitrates and switches grant, and the egress buffer presents the correct final word. pass_count reaches 2 of 2 checks with 0 fails.

6.5 Full System — End-to-End Through the Complete Pipeline



Figure 5: Complete system_top simulation. Four 512-bit words are driven into the external AXI4-Stream input; the waveform shows the resulting flits propagating through the link layer and PCIe-PHY layer, with all four lane symbol outputs converging on the correct, independently-computed golden value (0xCCCCCCCCCCCCCCCC). pass_count reaches 4 of 4 checks with 0 fails.

7. Summary of Limitations, Assumptions, and FutureImprovements

7.1 Assumptions

- A single, shared clock and active-low synchronous reset domain is assumed across the entire design; no clock-domain-crossing logic was required or implemented.
- The link layer's lack of a backpressure signal is treated as an intentional design characteristic of that module (it always accepts valid input every cycle) rather than a defect, and the protocol adapter was built to accommodate this rather than modify the link layer's interface.

- tkeep and tlast are not consumed past the AXI4-Stream domain boundary, since the link layer and PHY layer, as currently scoped, operate on fixed-size flits without a notion of partial-word validity or packet framing.

7.2 Known Limitations

- The interconnect is scoped to 2 inputs rather than a general N-input crossbar.
- The link layer's error-detection scheme is a simplified XOR-based checksum, not a production CRC-32 or Reed-Solomon FEC implementation.
- The protocol adapter introduces one idle 'bubble' cycle between consecutive 512-bit words for simplicity; this could be optimized to overlap with the next word's acceptance, at the cost of additional design and verification complexity.
- No synthesis, timing analysis, or physical implementation has been attempted; all results are functional RTL simulation only, as intended for Stage 1.

7.3 Planned Future Improvements (Stage 2)

- Generalize the interconnect to support additional input ports beyond the current 2-input instance.
- Replace the simplified XOR checksum with a standard CRC polynomial or a real FEC scheme.
- Optimize the protocol adapter to remove the idle bubble cycle between words.
- Proceed to synthesis using OpenROAD/OpenLane with an open-source PDK, followed by physical design through to GDSII.
- Demonstrate representative network packet-processing and machine-learning inference workloads end-to-end, per the Stage 2 requirements.